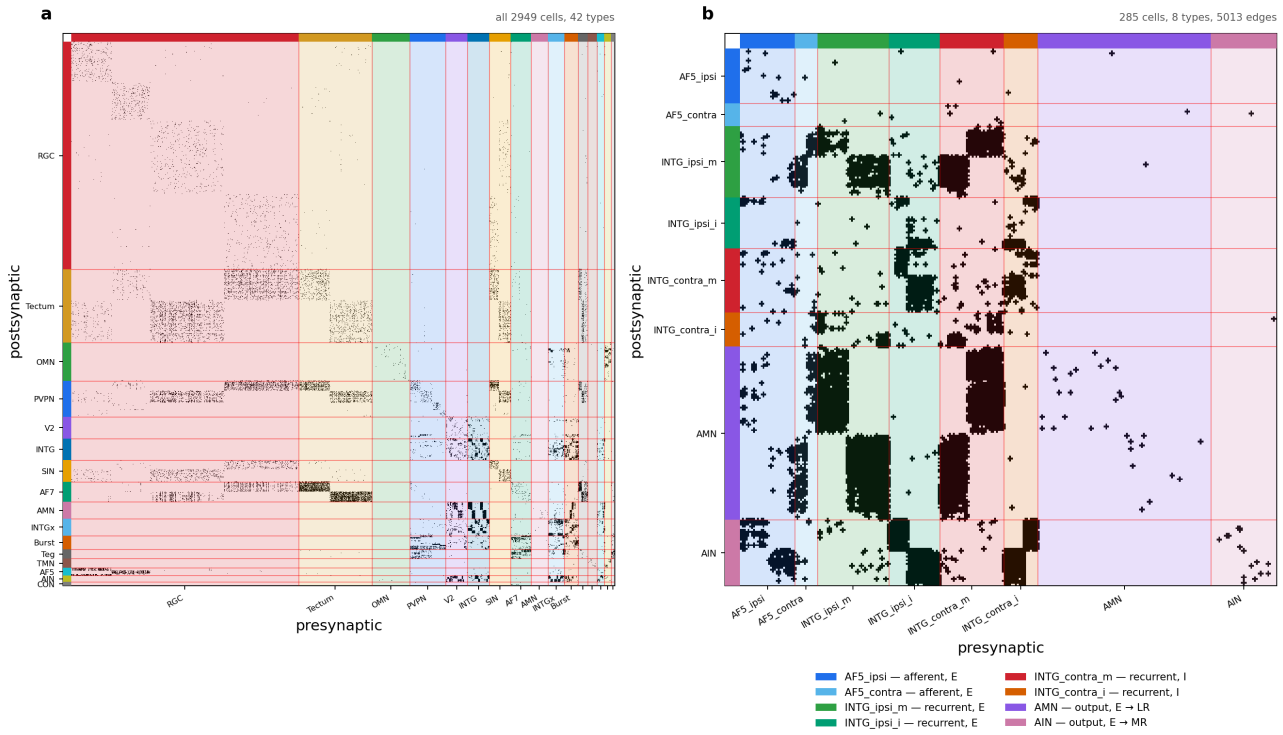
The larval zebrafish oculomotor integrator: circuit, task, and training data

2026-08-12

The larval zebrafish oculomotor integrator: circuit, task, and training data

Working document for the `feat/oculomotor` branch. It records what the circuit is taken to be, what the network will be asked to compute, and how the training data is generated. Everything here that is a *claim about biology* rather than a fact read off the reconstruction is marked as such — those are the parts that need the circuit owner's signature before any result means anything.

Source reconstruction: `Oculomotor_sortedData_081126.pkl`, 2949 cells, 42 cell types, 44,584 edges (density 0.51 %). Selected sub-circuit: `config/zebrafish/zebrafish_om_intg_285_v1.yaml`, 285 cells, 8 types.



The oculomotor reconstruction and the sub-circuit modelled here. (a) All 2949 cells, ordered by the 16 coarse cell-type families, support mask of the synapse-area matrix. (b) The 285-cell sub-circuit, ordered afferent then recurrent then output, and within a type left hemisphere before right. Colour carries the biology: blue = afferent, green = excitatory recurrent, red/orange = inhibitory recurrent, purple/pink = motor output. Rows are postsynaptic, columns presynaptic. Regenerate with `python scripts/plot_oculomotor_connectome.py -config config/zebrafish/zebrafish_om_intg_285_v1.yaml -out figures/zebrafish/fig_oculomotor_circuit.png`.

1. The circuit

1.1 Three pools

The 285 cells divide into an input stage, a recurrent integrator and a motor output stage.

pool	types	cells	Dale sign
afferent	`AF5_ipsi` (29), `AF5_contra` (12)	41	excitatory
recurrent	`INTG_ipsi_m` (38), `INTG_ipsi_i` (27), `INTG_contra_m` (34), `INTG_contra_i` (18)	117	ipsi excitatory, contra inhibitory
output	`AMN` (92), `AIN` (35)	127	excitatory

Within the sub-circuit there are 5013 edges, a density of 6.2 % — an order of magnitude above the 0.51 % of the full reconstruction. That concentration is what one expects of a recurrent integrator core plus its dedicated input and output stages, and it is the first (weak) evidence that the selection is a circuit rather than an arbitrary subset.

1.2 AF5 is an afferent, and the wiring says so

The two AF5 populations are the pretectal arborization field that carries optokinetic drive. Their afferent status is not assumed. In the measured matrix, AF5 sends 88.2 % of its outgoing synaptic area into the INTG/AMN/AIN core and receives 1/178th as much back (6.89e5 out against 3.87e3 in). This is the same asymmetry test that confirmed the matrix orientation using the retinal ganglion cells, which are 15.4x more outgoing than incoming.

Both AF5 types are left/right balanced — `AF5_ipsi` 15 L / 14 R, `AF5_contra` 6 L / 6 R — so a bilateral input gate is expressible. This is worth stating because it is not generally true in this dataset: `RGC_AF7` is reconstructed in the left hemisphere only (804 L / 0 R) and could not support a symmetric gate.

1.3 What drives the afferents — CLAIM, and an open one

The direction selectivity below is the circuit owner's description, not a measurement in this file. The combination rule is explicitly unresolved.

- `AF5_ipsi` — the left pool is driven by rightward-eye motion that is

leftward and/or forward; the right pool by leftward-eye motion that is rightward and/or forward.

- `AF5_contra` — the left pool is driven by rightward-eye motion that is

rightward and/or backward; the right pool by leftward-eye motion that is leftward and/or backward.

Whether the two conditions combine as AND, OR or XOR is **not known**. This is not a detail to be settled by a default: it decides whether a single afferent unit reports a conjunction (a narrow direction tuning) or a disjunction (a broad one), and therefore how much of the direction computation the recurrent circuit has to do itself. Until it is resolved, the input model should carry it as an explicit switch and the three variants should be trained and compared.

1.4 The integrator, and why the E/I split is by laterality

The four INTG types are the velocity-to-position integrator. The sign assignment follows their projection laterality: **ipsilateral projections excitatory, contralateral projections inhibitory**. That is the classical integrator motif — two half-integrators, one per hemisphere, each self-exciting locally and inhibiting its opposite number across the midline. Mutual inhibition is what lets the pair hold a *signed* position variable on a single positive-rate substrate.

This is a Dale assignment by cell type, exactly as in the heading-direction circuit, where `_ZHD_INH_PREFIXES` in `connectome_loaders.py` encodes the claim that the r1pi/dIPN cells are GABAergic. Neither is a measurement.

The difference is that here the claim is written in the config, per type, where it can be reviewed and flipped, rather than in a tuple of string prefixes inside a loader.

1.5 The output stage, and one simplification worth flagging

Only the **horizontal** pair of extraocular muscles is modelled:

- `AMN` drives the **lateral rectus** (`LR` in the plant), which abducts the

eye;

- `AIN` drives the **medial rectus** (`MR`), which adducts it.

Muscle keys refer to the six-muscle soft-body eye in `Plexus/prototype/eye`, where each muscle carries an innervation state `muscle.act` and the globe's pose is recovered by a Kabsch fit (`eye_pose` -> horizontal, vertical, torsion in degrees). Coupling the readout to that plant, rather than to an abstract scalar, is what would make "eye position" a mechanical quantity instead of a fitted one.

The simplification. `AIN` are abducens *internuclear* neurons. In vivo they are not motor neurons: they project across the midline to the oculomotor nucleus (`OMN`), whose motor neurons innervate the medial rectus. `OMN` is 207 cells in this reconstruction and is **not** in the selected pool. Treating `AIN` as driving the medial rectus therefore collapses a two-synapse pathway into one, and drops the sign inversion and delay that the missing synapse would contribute. This is a legitimate modelling choice, but it should be stated in any write-up, and the obvious control is to add `OMN` to `circuit.cell_types` and check whether the fitted dynamics change.

1.6 What is not in the pool

Two omissions are deliberate and one of them constrains the task:

- `Burst_T1A/T1B/T8/T9/T10` (74 cells) are the saccadic burst

generators — the source of the brief velocity command that a saccadic integrator integrates. They are excluded, so in the current pool the only anatomically defined input is the optokinetic AF5 drive. A saccade-driven task would have to inject velocity directly into INTG, which is a modelling choice, not a connectome-derived pathway.

- `OMN` (207 cells), as described in §1.5.

2. The task

2.1 What the network computes

The oculomotor integrator converts an eye-**velocity** command into an eye- **position** command: the motor neurons must hold a rate proportional to position long after the velocity signal has gone. Written as the leaky integral of the drive,

dP/dt = -P/tau + v(t)

the quantity of interest is `tau`, the time constant over which position leaks back to centre. A perfect integrator is `tau = infinity`; the biological integrator is finite, and measuring what `tau` the connectome-constrained circuit can support is the point of the exercise.

2.2 It needs no new task code

This is already a mode of the existing swim-integration generator. Setting

task.swim_integration.target_kind: scalar_xi task.swim_integration.xi_tau_s: <the leak> training.task_-targets: [translation]

yields a **1-input / 1-output** problem: the input channel is the velocity drive, the target is `xi = integral of v dt` with leak `xi_tau_s`. The `_integrate_leaky` helper in `graph_data_generator.py` implements exactly the equation above (forward Euler, `alpha = 1 - dt/tau`), and `tau <= 0` recovers the perfect integrator.

What the existing machinery does **not** provide is the neuron binding: which cells the input enters and which cells the readout reads. In the heading circuit both are hardwired to the HD taxonomy — the encoder through `graph_model.velocity_gate`, the decoder through `output_from_dipn_only` and a positional `[:n_bump]` slice. Redirecting them to the AF5 afferents and the AMN/AIN outputs is the real work, and it is Procedure B in `HOWTO_add_zebrafish_circuit.md`.

2.3 Saccades versus optokinetic drive

The two natural stimulus regimes differ, and the choice interacts with §1.6:

- **Optokinetic**: sustained whole-field motion, the drive AF5 actually

carries. Slow, continuous velocity — the regime the selected pool supports anatomically.

- **Saccadic**: brief high-velocity pulses separated by fixations, the

classical integrator probe. This is the regime the burst generators serve, and they are not in the pool.

The swim generator's stimulus is a Poisson train of boxcar impulses (`swim_rate_hz`, `swim_duration_s`) — structurally a saccade train already. So the saccadic regime is reachable by reparameterising the existing generator rather than writing a new one; what is missing is anatomy for the input, not code.

2.4 The open-loop problem

Coarsely: the circuit is asked to keep a moving target centred on the fovea while being told only how fast the target is moving, never where it is. That is the situation the anatomy puts it in. AF5 is a motion-sensitive arborization field — it reports optokinetic *velocity* — and nothing in the selected 285-cell pool reports absolute target position. A controller with position feedback can always correct, so its errors do not accumulate; a controller given velocity alone must reconstruct position by integration and has nothing to check the result against. Drift is therefore not a failure of the circuit but a property of the problem, and the meaningful question is not whether the target is held but for how long.

More specifically, write the target position `p(t)`, the gaze `g(t)`, and the retinal error `e = p - g`. Every trial starts foveated at the centre, `g(0) = p(0) = 0`. The plant is a rate control: the motor command sets gaze *velocity*, so gaze position is its integral. The ideal controller is then

dg/dt = v(t) , v = dp/dt -> e(t) = 0 for all t

and tracking is exact. The interesting cases are the three ways a biological integrator departs from it, each of which produces a different growth law for `|e|` — which is what makes them separable from a single measured trace.

Gain. If the velocity-to-position conversion is miscalibrated by a factor `k`,

dg/dt = k v(t) -> e(t) = (1 - k) [p(t) - p(0)]

The error is proportional to the *displacement* from the start, not to the distance travelled, because integration is linear. In a bounded workspace displacement is capped by the arena, so this error is self-limiting: in our geometry the target reaches a median 1.16 units from the centre, and `|1-k|` must exceed 0.52 before the target can leave a 0.6-unit fovea at all. A realistic miscalibration of a few percent is invisible. Gain is not what loses the target.

Leak. A real integrator is imperfect and relaxes toward its rest state with a time constant `tau`:

dg/dt = -g/tau + v(t)

In the frequency domain this is a *high-pass* on target position, corner frequency `1/tau`. Sustained excursions are under-represented, so the error saturates rather than growing without bound, and gaze is a shrunken, centre-biased copy of the truth. The counterintuitive consequence is that *slow* targets are lost first, because their motion is exactly the low-frequency content the leak removes. Measured: at `tau = 2 s` a fast target is never lost within 20 s while a slow one goes at 5.9 s, and the slow case needs `tau >= 8 s` to survive. `tau` is the quantity the INTG pool exists to make long, and this is the curve that says how long is long enough.

Noise. If the integrated velocity carries a white perturbation of intensity `sigma`,

dg/dt = v(t) + sigma * xi(t) , <xi(t) xi(t')> = delta(t - t')

the error is a random walk: `std[e(T)] = sigma sqrt(T)` per axis, unbounded but with zero mean. It is the only one of the three that cannot be corrected by better calibration and the only one that averages away across trials, so it is invisible to any analysis that pools trials before measuring drift.

Three growth laws — bounded-linear, saturating, and square-root — over one shared quantity. A drift trace measured from the real circuit can therefore be *classified*, not merely reported, which is the point of setting the problem up this way. `prototype/dot_tracking/openloop.py` implements the gain and leak cases and measures the survival time `t_lose`, the first moment `|e|` exceeds the fovea; the noise case is stated here for completeness but is not in the prototype, since a zero-mean drift is better measured against recorded data than against a simulation of itself.

3. The training dataset

3.1 Where it comes from

Task data is generated by `GNN_Main.py -o generate <config>`, which reaches `data_generate` -> `data_generate_task` (dispatch on `task.task_type`) -> `_generate_swim_integration_task`, all in `src/connectome_gnn/generators/graph_data_generator.py`.

The critical property: **generation is connectome-agnostic**. The generator writes `stimulus.zarr` of shape (trials, T, channels) and `target.zarr` of shape (trials, T, columns) — pure time series, with no notion of a neuron. The circuit appears only as a `circuit_provenance.json` written beside the splits, recording the circuit name, cell count and the SHA-256 of `J_effective`, so a dataset can later be checked against the circuit a checkpoint was trained on.

The consequence for planning: the dataset can be generated and inspected **before** the circuit registry entry, the connectome CSVs or the E/I assignment exist. That is the cheapest next step on this branch.

3.2 Parameters that define it

parameter	meaning
`n_trials_train` / `n_trials_test`	independent trials per split
`n_steps`, `dt`	samples per trial and the timestep, so trial duration is `n_steps * dt`
`swim_rate_hz`	mean Poisson rate of impulse onsets — the saccade rate
`swim_duration_s`	boxcar width of one impulse — the saccade duration
`forward_vel_mean` / `forward_vel_std`	amplitude distribution of the velocity drive
`target_kind`	which target is assembled; `scalar_xi` is the integrator
`xi_tau_s`	the leak; absent or <= 0 gives a perfect integrator
`seed`	stimulus RNG; fixed across baseline and comparison runs

`training.task_targets` then projects the on-disk superset down to the columns a given run trains on, so one dataset serves several sub-tasks.

3.3 A caution about the column layout

The mapping from `task_targets` to input/output dimensions is duplicated across **13 files** — `_TT_DIMS` in both the RNN and GNN model classes, and `_PROFILE_BY_TARGET` in `graph_trainer`, `graph_tester`, `plot_cx` and six figure scripts. A new task mode added in one place does not error elsewhere; it silently mis-slices columns. Any new mode has to be added to all of them, or the analysis figures will quietly describe the wrong variable.

4. Progress, 11 August 2026

Opened the branch `feat/oculomotor` and put the zebrafish configs back under version control — 254 of them had been absent from every worktree since the `config/zebrafish` bind-mount was commented out of `devcon-`

tainer.json, which left no figure script able to resolve a run config. Traced how the 917-cell heading-direction circuit was actually built (the colleagues' pickle, not the neuPrint query the filenames suggest) and wrote that up as HOWTO_add_oculomotor_circuit.md, since the same five inputs are needed again here. Read the new Oculomotor_sortedData_081126.pkl: 2949 cells, 42 types, 5 keys, no per-cell ordering variable; confirmed its matrix orientation empirically rather than assuming it. Selected the sub-circuit in two steps — first the 244-cell INTG/AMN/AIN core, then 285 cells once AF5 was added, which gave the pool the afferent stage it had been missing. Introduced CellTypeSpec so each type's pool, Dale sign, L/R split and role live in the config where they can be reviewed, rather than in hardcoded prefix tuples inside the loader. Rendered the two-panel connectivity figure above, and wrote this note.

Nothing has been trained, and no connectome CSVs exist yet. The five decisions below are what stand between this document and a first run.

5. What has to be decided next

1. **The AF5 combination rule** (§1.3) — AND, OR or XOR. Blocks the input model.
2. **Whether OMN joins the pool** (§1.5) — decides whether AIN -> medial

rectus is one synapse or two.

1. **Whether Burst_* joins the pool** (§1.6) — decides whether the saccadic

regime has an anatomical input or an injected one.

1. **The readout convention** — scalar eye position, or innervation of the

LR/MR pair coupled to the soft-body plant.

1. **The target leak xi_tau_s** — a free parameter, or the quantity to be

fitted against recorded eye position.